

Факултет за информатички науки и компјутерско инженерство

Елаборат за проектна задача по предметот

## **Континуирана Интеграција и Испорака**

Тема:

**REST API апликација - Систем за менаџирање со курсеви  
Courses Management System**

Професори: проф. д-р Милош Јовановиќ,  
проф. д-р Панче Рибарски

Изработка: Халид Веапоски , ИД: 186120

Датум: 05.02.2023

## Апликација

Апликацијата е REST API за менаџирање со курсеви, изработена во SpringBoot и Java. Овозможува CRUD операции за курсеви, студенти, наставници и оценки. Користи PostgreSQL за складирање на податоците и JPA, Hibernate ORM за комуникација со базата. Организирана е во контролери за секој дел од функционалноста и имплементира API endpoints. Користи ConfigMap и Secret во Kubernetes за управување со environment variables и лозинки. Обезбедува ефикасно менаџирање на податоците за курсевите, студентите и наставниците преку user-friendly интерфејс со помош на Thymeleaf (Spring Template Engine)

*CourseController*: Овој контролер управува со операции поврзани со курсевите. Од прикажување на листата со курсеви, додавање нов курс, ажурирање и бришење на постоечки курсеви.

Endpoint-и:

- /courses/allCourses (GET) - Прикажува листа со сите курсеви.
- /courses/addCourse (GET) - Прикажува форма за додавање на нов курс.
- /courses/add (POST) - Зачувува нов или ажурира постоечки курс.
- /courses/edit/{id} (GET) - Прикажува форма за ажурирање на курс со дадено ID.
- /courses/delete/{id} (GET) - Бришење на курс со дадено ID.
- /courses/paging (GET) - Прикажува курсеви со пагинација.

*GradeController*: Контролерот за оценки управува со прикажување и зачувување на оценки за студенти на одреден курс.

Endpoint-и:

- /grades/allGrades (GET) - Прикажува листа со сите оценки.
- /grades/addGrade (GET) - Прикажува форма за додавање на нова оценка.
- /grades/saveGrade (POST) - Зачувува нова оценка.
- /grades/studentEnrollmentSummary (POST) - Прикажува содржина за уписот на студент во курс.
- /grades/CourseEnrollmentSummary (GET) - Прикажува форма за избор на студент за упис во курс.

*StudentsController*: Овој контролер се однесува кон управување со студенти, нивното прикажување, додавање и бришење.

Endpoint-и:

- /students/allStudents (GET) - Прикажува листа со сите студенти.
- /students/addStudent (GET) - Прикажува форма за додавање на нов студент.
- /students/save (POST) - Зачувува нов студент.
- /students/studentEnrollmentSummary (POST) - Прикажува содржина за упис на студент во курс.
- /students/CourseEnrollmentSummary (GET) - Прикажува форма за избор на студент за упис во курс.

*TeacherController*: Контролерот за наставници управува со операции како прикажување, додавање, ажурирање и бришење на наставници.

Endpoint-и:

- /teachers/allTeachers (GET) - Прикажува листа со сите наставници.
- /teachers/addTeacher (GET) - Прикажува форма за додавање на нов наставник.
- /teachers/add (POST) - Зачувува нов или ажурира постоечки наставник.
- /teachers/delete/{id} (GET) - Бришење на наставник со дадено ID.
- /teachers/edit/{id} (GET) - Прикажува форма за ажурирање на податоци за наставник со дадено ID.

Апликација за менаџирање со курсеви користи PostgreSQL база на податоци за складирање и организација на информациите. Оваа релациона база на податоци им овозможува на корисниците ефикасно да пребаруваат, додаваат, ажурираат и бришат информации поврзани со курсевите, студентите и наставниците во системот. За оваа цел, апликацијата користи JPA и Hibernate ORM за комуникација со базата на податоци.

Карактеристики на PostgreSQL базата на податоци во контекстот на апликацијата:

*Табела за курсеви:* Име на табелата: course

Колони: id (примарен клуч), name (име на курс), description (опис на курс), type (тип на курс), teacher\_id (референца кон наставник)

*Табела за студенти:* Име на табелата: students

Колони: username (примарен клуч), name (име на студент), surname (презиме на студент), password (лозинка)

*Табела за оценки:* Име на табелата: grade

Колони: id (примарен клуч), value (вредност на оценката), student\_id (референца кон студент), course\_id (референца кон курс), date (датум на додавање на оценката)

*Табела за наставници:* Име на табелата: teacher

Колони: id (примарен клуч), name (име на наставник), surname (презиме на наставник), date\_of\_employment (дата на вработување)

Оваа структура на табелите и нивните колони го обезбедува ефикасното и организирано чување на податоците поврзани со курсевите, студентите и наставниците. Hibernate ORM служи како помошник за лесна и сигурна комуникација со PostgreSQL базата на податоци, обезбедувајќи ефикасен и модуларен пристап до податоците. Со користење на PostgreSQL, апликацијата гарантира сигурност, интегритет на податоците и перформанси при работа со голем број на записи во базата.

Основната идеја зад Dockerfile и Docker Compose конфигурацијата е да се контејнеризира Spring Boot апликацијата и PostgreSQL базата на податоци за лесна и пренослива дистрибуција. Според тоа:

Се одбира base image со JDK 17 за Java апликацијата. Потоа се дефинира аргумент за JAR фајлот што ќе се копира. Со COPY командата се копира JAR фајлот во контејнер и го преименува како lab-0.0.1-SNAPSHOT.jar

Следно ги отвара портите 9191 на контејнерот за комуникација И поставува стартувачка точка(entrypoint) за SpringBoot апликацијата.

GitHub репозиториум линк: <https://github.com/halid-ux/Courses-Management-System>

Dockerfile link : <https://github.com/halid-ux/Courses-Management-System/blob/main/Dockerfile>

Docker-compose.yml link: <https://github.com/halid-ux/Courses-Management-System/blob/main/docker-compose.yml>

## GitHub Actions за Continuous Integration (CI)

За осигурување на континуирана интеграција (CI) на проектот, користам GitHub Actions. GitHub Actions е вграден систем за автоматизација на работни процеси, што овозможува автоматско извршување на тестови, изградба на апликацијата и други активности по секој нов commit на репозиториумот. Се користи GitHub Action-от од Docker кој се вика docker/build-push-action и се достапен на следниот линк: <https://github.com/docker/build-push-action>

GitHub Workflow за CI:

Во репозиториумот има еден дефиниран GitHub Workflow кој се активира при секој нов commit.

Линк до Github Workflow: <https://github.com/halid-ux/Courses-Management-System/blob/main/.github/workflows/main.yml>

## Kubernetes

За потребите на овој проект користената верзија на Kubernetes е Minikube . Креирани се потребните Kubernetes манифести за Namespace, Deployment за апликацијата, Service, Ingress, StatefulSet и Service за PostgreSQL базата на податоци, и ConfigMaps и Secrets за информациите за остварување на конекција до базата на податоци.

Линк до манифестите: <https://github.com/halid-ux/Courses-Management-System/tree/main/kubernetes>

### Namespaces

Во Kubernetes е важно да се користат Namespaces за организација и изолација на ресурсите. Во овој проект, имаме креирано именуван Namespace наречен kiii-project. Оваа изолација ќе овозможи лесно управување и организација на ресурсите поврзани со проектот.

Линк до манифестот за Namespace: <https://github.com/halid-ux/Courses-Management-System/blob/main/kubernetes/namespace.yaml>

## Deployment

Kubernetes манифести за конфигурација (ConfigMap) и деплојмент (Deployment) за апликацијата која користи Spring Boot и PostgreSQL база на податоци.

ConfigMap: Овој дел креира ConfigMap во Kubernetes што содржи конфигурациски параметри за Spring Boot апликацијата.

Конкретно, се поставуваат параметри како портата за серверот, активниот профил, параметри за поврзување со базата на податоци (URL, корисничко име, лозинка и др.).

Ова е начинот како Kubernetes апликацијата може да динамички промени конфигурација без потреба од пре-компилација на кодот.

Deployment (kiii-springboot-deployment):

Овој дел дефинира Deployment објект во Kubernetes, што означува како и колку копии од контејнерот за Spring Boot апликацијата треба да се подигнати.

Секој под (replica) користи конфигурациски параметри од ConfigMap преку мапирање на volume (kiii-springboot-configmap-volume), што и овозможува на апликацијата да пристапи до податоците.

Docker контејнерот користи Docker образ (image) halid99/java\_app:1.0.0, а аргументите (args) проследуваат командата за стартување на апликацијата.

Оваа конфигурација овозможува лесно менаџирање на конфигурацијата и скалирање на апликацијата на Kubernetes.

Линк до манифестот за Deployment: <https://github.com/halid-ux/Courses-Management-System/blob/main/kubernetes/deployment.yaml>

## Service & Ingress for Springboot app

- Service (kiii-project-springboot-service):

Овој дел креира Kubernetes сервис, кој служи како "load balancer" за пренасочување на трафикот кон подовите (replicas) од Spring Boot апликацијата.

Сервисот е именуван како kiii-project-springboot-service и припаѓа на namespace kiii-project. За дополнителна конфигурација, во анотациите се поставени анотации за ингрес, но се закоментирани (#), бидејќи сега се применува алтернативен начин за пренасочување.

- Ingress (kiii-project-ingress):

Овој дел креира Kubernetes ингрес, што означува како трафикот треба да се пренасочува кон сервисот во специфицираниот namespace.

Ингресот овозможува пристап до апликацијата преку поставената доменско име (kiii-project.local) и префиксот на патот ("/").

Анотациите служат за дополнителна конфигурација на ингресот. Во овој случај, анотацијата за пренасочување (rewrite-target) е закоментирана, но може да се откоментира доколку има потреба за пренасочување на патот.

Оваа конфигурација овозможува пристап до Spring Boot апликацијата преку специфицираниот domain name и пат на ингресот, со трафик пренасочен преку Kubernetes сервисот.

Линк до манифестот: <https://github.com/halid-ux/Courses-Management-System/blob/main/kubernetes/ingress.yaml>

## StatefulSet & Service for Postgresql

Овој YAML документ опишува конфигурација во Kubernetes за PostgreSQL база на податоци. Применети се концепти како ConfigMap, Secret, PersistentVolumeClaim, StatefulSet и Service.

ConfigMap дефинира конфигурациски параметри за PostgreSQL, вклучувајќи име на базата (POSTGRES\_DB) и корисничко име (POSTGRES\_USER).

Secret чува лозинката за PostgreSQL, заштитена со типот Opaque и енкодирана во Base64. PersistentVolumeClaim (kiii-postgres-vol) резервира простор за податоци со StorageClass local-path, поддржуван од minikube.

Ова обезбедува перзистентен складишен простор за податоците на PostgreSQL.

StatefulSet (kiii-postgres) определува PostgreSQL под со репликација 1, користејќи ConfigMap и Secret за конфигурација и лозинка. Секој под задржува перзистентен обем за податоците. Service (kiii-postgres-svc) дозволува пристап до PostgreSQL подот и го насочува на порта 5432.

Линк до манифестацијата StatefulSet & Service: <https://github.com/halid-ux/Courses-Management-System/blob/main/kubernetes/postgres-service.yaml>